

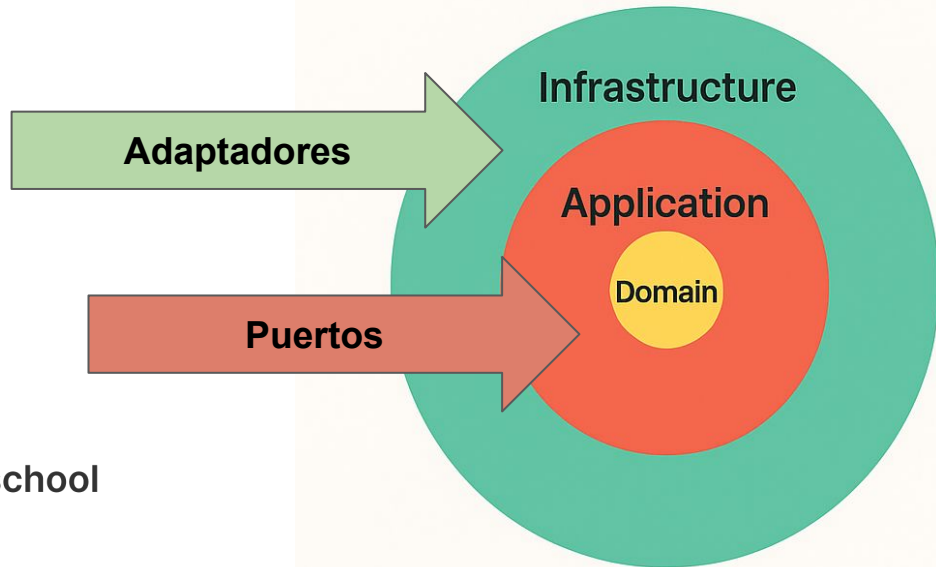
MÓDULO 2: ARQUITECTURA DE SOFTWARE

PUERTOS Y ADAPTADORES (INTERFACES + IMPLEMENTACIÓN)

APLICANDO CLEAN ARCHITECTURE CON TYPESCRIPT

Puertos y Adaptadores

Los **puertos** expresan **necesidades**; los **adaptadores** expresan **tecnología**.



Antipatrones

Adaptadores que **devuelven entidades** (fuera de *application*).

Puertos que **filtran** detalles técnicos (SQL/HTTP) al caso de uso.

Controladores que **contienen lógica de negocio**.

Repos que **mutan** DTOs o estados ocultos.

No tener **tests de contrato** (cada refactor rompe algo distinto).



IA como Copiloto

Generar test de contrato

“Dado este puerto {OrderRepository}, genera una suite de tests de contrato en Vitest que valide guardar/leer, idempotencia de save y atomicidad. Devuélvela parametrizable por fábrica.”

Diseñar mapeadores

“A partir del snapshot del agregado `Order`, escribe funciones puras `toRows(snapshot)` y `fromRows(rows)` y casos de borde. No uses librerías externas.”

Revisión de acoplamientos

“Revisa este conjunto de adaptadores y señala cualquier acoplamiento accidental al dominio o a la capa de aplicación. Sugiere límites y nombres coherentes.”

Outbox + dispatcher

“Genera SQL para tabla `outbox` y un job Node.js que lea, publique (simulado) y marque `published_at`, con reintentos exponenciales.”

Controlador HTTP

“Dado este caso de uso {AddItemToOrder}, crea un controlador Fastify que valide input con zod y mapee errores tipados a HTTP 400/404/409/503.”